CEMAC MONETARY INTELLIGENCE

Data Pipeline — README

Documentation technique complète · Réutilisation · Travail collaboratif

Python 3.10+	pandas / numpy	openpyxl	64 trimestres	0 NaN modèles

1. Vue d'ensemble du projet

Ce projet constitue la couche Data Engineering d'un système intelligent d'aide à la programmation monétaire pour la BEAC (Banque des États de l'Afrique Centrale), avec le Cameroun comme pivot structurel de la zone CEMAC (43% du PIB régional).

Le pipeline transforme 9 fichiers Excel BEAC bruts en 4 datasets ML-ready couvrant 64 trimestres (2009-Q1 → 2024-Q4), destinés à alimenter trois modèles économétriques : BVAR, HMM et New Keynesian 3 équations.

Architecture en couches

Couche	Composant	Description
L0 — Sources	9 fichiers Excel BEAC	IPC, PIB, M2, Réserves, TOFE, TIAO, Inflation CEMAC, FRED API
L1 — Staging	Routing.sh	Dispatch automatique par préfixe vers datalake/raw/{prefix}/
L2 — Raw	datalake/raw/beac/	Données brutes non transformées, archivées
L3 — ETL	7 scripts pipeline/	Nettoyage, raccordement de séries, variables dérivées
L4 — Data Lake	3 étapes d'assemblage	assemble → merge → prepare → macro_global_v2.csv (64×39)
L5 — ML Ready	bvar / hmm / nk_ready.csv	0 NaN, stationnarité vérifiée, directement consommables

2. Prérequis et installation

Environnement Python

Python 3.10 ou supérieur est requis. Les dépendances minimales sont :

```
pip install pandas numpy openpyxl statsmodels scipy
```

Pour les variables exogènes FRED (optionnel) :

```
pip install fredapi
```

Structure des fichiers sources requis

Avant de lancer le pipeline, placer les fichiers Excel dans datalake/raw/beac/ avec exactement ces noms :

Nom de fichier attendu	Contenu
IPC_CMR.xlsx	Indice des Prix à la Consommation — Cameroun
COMPTES_NAT_CMR.xlsx	Comptes nationaux trimestriels (PIB courant + réel base 2016)
STAT_MONETAIRE_CMR.xlsx	Statistiques monétaires et financières (M2, crédit, avoirs)
RESERVES_CMR.xlsx	Réserves de change CEMAC (variation trimestrielle)
TOFE_CMR.xlsx	Tableau des Opérations Financières de l'État
TIAO_CMR.xlsx	Taux directeur BEAC (TIAO) trimestriel
Base_inflation_CEMAC.xlsx	IPC glissement annuel — 6 pays CEMAC
Reserves_1993_2008.xls	Réserves CEMAC mensuelles 1993-2008 (legacy, optionnel)

⚠ Important : les fichiers BEAC sont souvent livrés avec des noms contenant des espaces et des accents. Renommer les fichiers exactement comme indiqué ci-dessus avant de lancer le pipeline.

---

3. Lancement rapide

Commande unique

Depuis la racine du projet (là où se trouve config.py) :

```
python run_pipeline.py
```

Le pipeline s'exécute en 11 étapes, produit les logs dans logs/ et affiche un rapport final avec le statut de chaque étape et les dimensions des fichiers produits.

Commandes disponibles

Commande	Effet
python run_pipeline.py	Pipeline complet, étapes 1 → 11
python run_pipeline.py --step 7	Reprendre à partir de l'étape 7
python run_pipeline.py --only etl	Relancer uniquement les ETL BEAC
python run_pipeline.py --dry-run	Simuler sans rien exécuter
python run_pipeline.py --check	Vérifier les fichiers sources seuls
python run_pipeline.py --no-routing	Ignorer Routing.sh (fichiers déjà en place)
python config.py	Afficher la configuration et valider l'environnement

Résultats attendus

À l'issue du pipeline, les fichiers suivants sont produits dans datalake/data/final/ :

- macro_global_v2.csv — 64 lignes × 39 colonnes — dataset maître complet

- bvar_ready.csv — 60 obs × 9 colonnes — 0 NaN — période 2010-Q1 → 2024-Q4
- hmm_ready.csv — 64 obs × 5 colonnes — 0 NaN — variables standardisées z-score
- nk_ready.csv — 63 obs × 11 colonnes — 0 NaN — période 2009-Q2 → 2024-Q4

---

4. Description du pipeline — 11 étapes

Étape 1 — Routing.sh

Rôle : dispatcher automatique. Lit tous les fichiers du dossier staging/, extrait le préfixe avant le premier underscore, crée le sous-dossier correspondant dans datalake/raw/ et déplace le fichier.

```
staging/beac_IPC_CMR.xlsx  →  datalake/raw/beac/IPC_CMR.xlsx
```

Étapes 2 à 6 — ETL BEAC internes

Script	Sortie	Variables produites
transform_ipc.py	ipc.csv 68×3	ipc_index, inflation (YoY %) — raccord base 2011→2022
transform_comptes_nat.py	comptes_nat.csv 68×6	gdp_current, gdp_real, gdp_growth, gdp_deflator
transform_monetaire.py	monetaire.csv 68×7	m2, m2_growth, credit, credit_growth, avoirs_ext_nets
transform_reserves.py	reserves.csv 61×5	fx_reserves, fx_reserves_yoy, reserves_flag (YoY>30%)
transform_tofe.py	tofe.csv 68×7	fiscal_balance, oil_revenue_share, gov_revenue_hd

Étapes 7 à 8 — ETL sources BEAC CEMAC

transform_manual_series.py fusionne 3 sources BEAC régionales :

- TIAO_CMR.xlsx (sheet TIAOTRI) → beac_rate trimestriel, 5/5 validations exactes contre références BEAC
- Base_inflation_CEMAC.xlsx → inflation CEMAC pondérée (CM 43%, TD 18%, GA 14%, CG 11%, GQ 10%, CF 4%)
- RESERVES_CMR.xlsx (Reserves_change) → reserves_cemac en Mrd XAF

Sortie : manual_series.csv — 68 obs × 7 colonnes incluant delta_beac, log_reserves_cemac, dlog_reserves_cemac.

Étape 9 — ETL Externe FRED (optionnel)

Nécessite une clé API FRED (gratuite sur fred.stlouisfed.org). Télécharge 6 séries internationales : FEDFUNDS, ECBDFR, DEXUSEU, DCOILWTICO, VIXCLS, CHNGDPNQDSMEI. Sans clé, le pipeline continue sans ces 4 variables exogènes BVAR.

```
export FRED_API_KEY=your_key_here  # Linux/Mac
set FRED_API_KEY=your_key_here     # Windows
```

Étape 10 — Assemblage (assemble_model_data.py)

Fusionne les 5 CSV BEAC internes sur la colonne date. Calcule delta_inflation = diff(inflation), log_fx_reserves, dlog_fx_reserves, dummies 2016-Q4 et COVID. Valide 4/4 cross-checks contre FMI/INS/BEAC. Sortie : macro_model_ready_v3.csv — 64×27.

Étape 11 — Fusion globale (merge_global.py)

Fusionne macro_model_ready_v3 + manual_series + external (si disponible). Sortie : macro_global_v1.csv — 64×33 colonnes.

Étape 12 — Préparation ML (prepare_model_v2.py)

Applique 4 transformations finales :

- T1 — Imputation : 3 NaN réserves (2009-Q1/Q2/Q3) → interpolation linéaire, reserves_flag=1 conservé
- T2 — Output gap : filtre Hodrick-Prescott λ=1600 sur gdp_real → gdp_gap_hp (ADF t=-6.27 *** I(0))
- T3 — Gap inflation : inflation_gap = inflation − 3.0 (cible BEAC Statuts art.1)
- T4 — Standardisation : z-score μ=0 σ=1 sur 4 variables HMM (évite que oil_revenue_share écrase log_fx)

Extrait ensuite les 3 sous-ensembles ML-ready. Sortie finale : macro_global_v2.csv — 64×39 + bvar/hmm/nk_ready.csv.

---

5. Datasets ML-ready — spécifications

bvar_ready.csv — Bayesian Vector Autoregression

Propriété	Valeur
Dimensions	60 obs × 9 colonnes — période 2010-Q1 → 2024-Q4
NaN	0 — données complètes sur toutes les variables
Ratio T/k	12 — suffisant pour estimation BVAR avec prior Minnesota
Variables endogènes (5)	delta_inflation · gdp_growth · m2_growth · dlog_fx_reserves · fiscal_balance
Variables exogènes (2+)	dummy_2016q4 · dummy_covid · [brent_yoy · eur_usd_yoy si FRED disponible]
Stationnarité	Toutes I(0) vérifiées par ADF : t entre -2.81 et -9.53
Question modèle	Quelles sont les interactions dynamiques entre agrégats ? Impact d'un choc Brent ?

hmm_ready.csv — Hidden Markov Model

Propriété	Valeur
Dimensions	64 obs × 5 colonnes — période 2009-Q1 → 2024-Q4
NaN	0 — après interpolation linéaire 2009-Q1/Q2/Q3
Variables (4 _std)	inflation_std · log_fx_reserves_std · gdp_growth_std · oil_revenue_share_std
Standardisation	z-score obligatoire — oil (σ=7.97) vs log_fx (σ=0.35) sans standardisation
Paramètres z-score	inflation (μ=2.76, σ=1.93) · log_fx (μ=8.64, σ=0.35) · gdp_growth (μ=3.72, σ=1.79)

Question modèle	Dans quel régime l'économie se trouve-t-elle ? Expansion / Tension / Crise ?

nk_ready.csv — New Keynesian 3 équations

Propriété	Valeur
Dimensions	63 obs × 11 colonnes — période 2009-Q2 → 2024-Q4
NaN	0 — toutes variables complètes
Éq. IS (demande)	gdp_gap_hp ~ beac_rate + fiscal_balance
Éq. Phillips (offre)	delta_inflation ~ gdp_gap_hp + cemac_inflation
Règle de Taylor (MP)	delta_beac ~ inflation_gap + gdp_gap_hp
Note stationnarité	beac_rate et inflation I(1) → utiliser delta_beac + inflation_gap dans Taylor
Question modèle	Comment la BEAC devrait-elle ajuster ses taux face aux chocs ?

---

6. Configuration centralisée — config.py

config.py est le fichier central du projet. Il contient tous les chemins et tous les paramètres économiques. Aucun script ne hard-code de chemin — ils importent tous depuis config.py.

Utilisation dans un script

```
from config import PATHS, PARAMS

# Lire un fichier source
df = pd.read_excel(PATHS['ipc'])

# Écrire une sortie
df.to_csv(PATHS['ipc_csv'], index=False)

# Utiliser un paramètre
inflation_gap = df['inflation'] - PARAMS['pi_star']   # pi_star = 3.0
```

Paramètres configurables

Paramètre	Valeur défaut	Description
period_start	2008-Q1	Premier trimestre chargé par les ETL
period_end	2024-Q4	Dernier trimestre — mettre à jour à chaque nouvelle livraison
bvar_start	2010-Q1	Début effectif BVAR (réserves disponibles YoY)
pi_star	3.0	Cible d'inflation BEAC (Statuts BEAC art. 1)
hp_lambda	1600	Paramètre filtre Hodrick-Prescott (standard trimestriel)

reserves_shock_threshold	30.0	Seuil YoY% pour reserves_flag (choc majeur)

7. Guide de travail collaboratif

Règle fondamentale — ne jamais modifier datalake/raw/

Les fichiers dans datalake/raw/ sont les données sources. Ils ne doivent jamais être modifiés. Toute transformation doit être scriptée dans pipeline/ et traçable dans les logs.

Ajouter une nouvelle source de données

- Placer le fichier dans staging/ avec le préfixe de la source : beac_NOM.xlsx, wb_NOM.csv, etc.
- Lancer Routing.sh — le fichier sera automatiquement déplacé dans datalake/raw/{prefix}/
- Créer un nouveau script pipeline/final_transform_NOM.py en copiant un script existant comme modèle
- Ajouter le chemin dans config.py (PATHS_RAW et PATHS_DATA)
- Ajouter l'étape dans run_pipeline.py avec ses dépendances
- Lancer python run_pipeline.py --only etl pour tester

Mettre à jour les données existantes (nouvelle livraison)

- Remplacer le fichier Excel dans datalake/raw/beac/ par la nouvelle version
- Vérifier que les noms de sheets n'ont pas changé (risque principal lors des mises à jour BEAC)
- Lancer python run_pipeline.py --step 2 pour relancer depuis les ETL
- Vérifier le rapport final — les dimensions et moyennes doivent rester cohérentes

Modifier un paramètre économique

Tous les paramètres sont dans config.py. Par exemple, pour changer la période d'analyse :

```
# config.py
PARAMS = {
    "period_end": "2025-Q4",   # ← modifier ici uniquement
    ...
}
```

Puis relancer python run_pipeline.py pour régénérer tous les datasets.

Conventions de nommage

Élément	Convention
Fichiers sources	NOM_COURT_CMR.xlsx ou NOM_SOURCE.xlsx — sans espaces, sans accents
Scripts ETL	final_transform_{source}.py dans pipeline/
CSVs intermédiaires	nom_court.csv dans datalake/data/beac/ ou external/
Datasets finaux	macro_global_vN.csv + {modele}_ready.csv dans final/
Logs	pipeline_YYYYMMDD_HHMMSS.log dans logs/

Variables	snake_case — préfixe delta_ pour différences, log_ pour logarithme

8. Bugs corrigés — changelog

Les problèmes suivants ont été identifiés lors de l'audit et corrigés dans la version actuelle :

ID	Fichier	Description et fix
B1-B8	`config.py`	Noms de fichiers dans PATHS_RAW ne correspondaient pas aux fichiers réels (espaces, accents, variantes). Fix : renommage des fichiers sources + mise à jour config.py
B9	`transform_manual_series.py`	BASE_IN + 'TIAO_-_Copie.xlsx' hardcodé — fichier réel est TIAO_CMR.xlsx. Fix : correction du nom
B10	`transform_manual_series.py`	sheet_name='Feuil2' inexistante dans RESERVES_CMR.xlsx — sheet réelle = 'Feuil1'. Fix : correction du nom
B11	`transform_manual_series.py`	np.log() sur colonne avec NaN → TypeError. Fix : pd.to_numeric(..., errors='coerce') avant np.log()
B12	`run_pipeline.py`	Étape 11 pointait vers 'final_prepare_model_v2.py' à la racine — fichier est dans pipeline/. Fix : correction du chemin
B13	`Tous scripts pipeline/`	ModuleNotFoundError: 'config' — scripts dans pipeline/ ne trouvaient pas config.py à la racine. Fix : injection sys.path.insert(0, ROOT)
B14	`Structure`	Dossiers datalake/data/beac/, external/, final/, logs/ absents → crash à l'écriture. Fix : création automatique par run_pipeline.py

9. Conformité à l'architecture globale du projet

L'architecture cible du projet complet comporte 5 couches. Le pipeline data constitue les 3 premières couches — celles qui sont implémentées.

Couches implémentées ✓

Couche	Statut	Détail
Ingestion / ETL	**✓ COMPLET**	Routing.sh + 7 scripts ETL + config.py + run_pipeline.py
Stockage (Data Lake)	**✓ COMPLET**	Raw / Data / Final — 12 fichiers CSV produits, 0 NaN sur variables modèles
Base ML	**✓ COMPLET**	bvar_ready · hmm_ready · nk_ready — stationnarité ADF vérifiée

Couches à développer

Modélisation	À faire	BVAR (statsmodels) + HMM (hmmlearn) + NK 3 équations (scipy)
Service / API	À faire	API Gateway FastAPI + Service d'Alertes (règle Taylor déviation > seuil)

Présentation	À faire	Dashboard Streamlit + Chatbot RAG (LangChain + ChromaDB)

Note sur le LSTM (intentionnellement exclu) : le LSTM nécessite généralement un minimum de 500 à 1000 observations pour converger. Avec 64 trimestres, le rapport signal/bruit serait défavorable et les résultats non fiables. Ce choix est justifiable scientifiquement en soutenance.

10. Recommandations pour la suite

Priorité 1 — Avant de coder les modèles

- Vérifier que les 3 fichiers ML-ready passent les tests de stationnarité avec statsmodels (ADF sur toutes les variables endogènes BVAR)
- Tester la corrélation croisée entre delta_inflation et gdp_growth pour confirmer la structure BVAR
- Ajouter la clé FRED_API_KEY pour les 4 variables exogènes BVAR (brent_yoy, eur_usd_yoy, delta_fed, delta_ecb)

Priorité 2 — Robustesse du pipeline

- Ajouter des tests unitaires minimaux (test_pipeline.py) : vérifier shape, NaN, plages de valeurs attendues
- Ajouter des checksums MD5 sur les fichiers sources pour détecter les mises à jour non déclarées
- Documenter la version des données dans un fichier data_manifest.json (date, source, nb lignes)

Priorité 3 — Architecture globale

- Commencer par le HMM (hmmlearn) — c'est le plus simple à implémenter et il fournit les régimes pour contextualiser les résultats BVAR et NK
- Enchaîner avec le BVAR (statsmodels VAR + prior Minnesota manuel ou pymc-bvar)
- Finir par le NK 3 équations (régression OLS sur chaque équation, puis système simultané GMM)
- Chaque modèle doit exposer une fonction predict(scenario_dict) → DataFrame pour l'API Gateway